

Tutoring 02

Git, Maven, Tomcat & Docker

Francesco L. De Faveri

Web Applications Tutoring
Academic Year: 2025-2026



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Outline

- Git
- Maven
- Tomcat
- Docker



General Information for HW1

Tools for ER schema, Logical schema or diagrams:

- <https://draw.io/>

Tools for mockups:

- <https://balsamiq.cloud/>
(30 days of free trial)

- <https://www.lucidchart.com/>
- <https://draw.io/>



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Git



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

GIT

Git is a Distributed Version Control System (DVCS) used for tracking changes in source code during software development.

Git allows:

- collaboration
- changes tracking
- code sharing

Key features:

- Version control
- Distributed development
- Branching
- Merging

If you need to download git: <https://git-scm.com/downloads>



Linus Torvalds



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Why GIT?

Why using git instead of a shared “standard” repository (e.g. Google Drive)?

- **Deep integration with both IDEs and Shell** → easy to use everywhere (even on servers)
- “Explicit” tracking of changes → easy to compare versions and revert errors

Therefore, **even if you work on a project on your own and you use multiple PCs it can be useful to use git.** (Learn it well for your future job!)



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

GIT & Bitbucket Repositories

Steps to clone a Bitbucket repository:

1. Open the desired repository
(e.g., the one that the professor assigned to your group)
2. Click on the clone button and copy the instruction
3. Choose the local folder where you want to store the cloned repository.
4. Open a new terminal in that folder and paste the copied instruction.

**** Windows users:** it is strongly suggested to execute git commands through **GIT BASH**. GIT BASH is a shell that is installed automatically when you install git.

Otherwise, use it through the
integration via your favourite IDE



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Cloning the TA Web App Repository



```
git clone https://francescodf99@bitbucket.org/frncl/tutor-wa-2025.git
Cloning into 'tutor-wa-2025'...
remote: Enumerating objects: 342, done.
remote: Counting objects: 100% (342/342), done.
remote: Compressing objects: 100% (257/257), done.
remote: Total 342 (delta 111), reused 166 (delta 36), pack-reused 0 (from 0)
Receiving objects: 100% (342/342), 10.86 MiB | 9.80 MiB/s, done.
Resolving deltas: 100% (111/111), done.
```



GIT - commit

Git comprises of 3 “components”:

Working Repository → The directory on your local computer where you are currently working with Git. It is the directory that contains the files and directories of your project, and it represents the current state of your project.

INDEX (Staging Area) → When you make changes to files in your working directory, those changes are not immediately committed to the repository. **They are first added to the staging area** where can be reviewed and organized. The staging area allows you to selectively add changes to the snapshot, rather than committing all changes at once. This is useful for when you want to commit only certain changes that are related to a specific feature or bug fix.

HEAD → A reference to the most recent commit in the current branch of the repository. It points to the commit that represents the current state of the repository. HEAD is essentially a pointer to a specific commit in the repository's commit history.



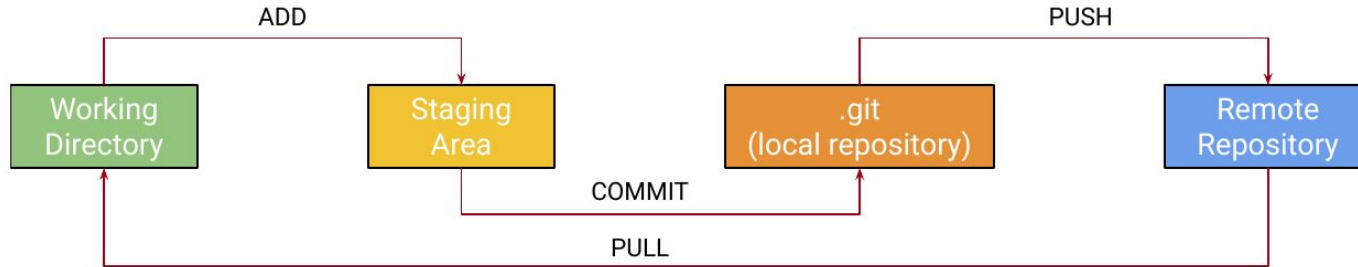
UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Git commit

The overall workflow is the following:



1. Up to the commit step, everything is local on your computer.
2. However, the commit operation takes the files as they are in the staging area and stores that snapshot permanently to your Git directory
3. Push actually uploads the files to the remote repository.
4. Pull synchronizes the content of your local repository with the remote repository



Branch

In your homework you will work in parallel with your colleagues, developing different features.

Sometimes it is useful to create separate branches (one for each major feature) to avoid interfering with your mate's work.

To do this:

git checkout -b <branchname> #creates a branch

git checkout <branchname> #switches to the branch

Remember: at the end of your project all the branches must be merged to the main branch.



Branch

git merge <branchname> #merges the branch <branchname> with the current one

git merge --abort #aborts the merge

If different people updated the same files on different branches, when you merge them conflicts arise. You will need to solve them

Branches can have a complex structure.

However there exist a **single main branch**. Pay attention when you create and merge branches.



Branch

Other commands:

git branch #lists all the branches

git branch <branchname> #creates a branch, same as “git checkout -b”

git branch -d <branchname> #deletes a branch LOCALLY

git push origin --delete <branchname> #deletes a branch REMOTELY



Pull

To align the content of your local repository with the remote repository (e.g., if others already did some work) you can use the command:

git pull #pulls every branch

If you work on separate branches (e.g., one for each group component) you can also pull single branches:

git pull origin <branchname> #pulls only the specified branch



Add

Once you created/modified some files you need to commit them and upload them.

1. ADD

- Add single files, they must be separated by single spaces:
git add <file_1> <file_2> ... <file_n> #adds single files
- Add files that are NOT at the same level of root:
git add /path/to/file/<filename> #add a single files in a folder
- Add all the files
git add . #adds all the changes



Commit

Step 2.

git commit -m “message” #commits the changes to ALL the added files (also before)

It is important to include a message on your commits. This allows to properly keep track of the changes.

TIP: avoid non-explanatory commits like “update”, “new files”, etc. It is better if the commits contains also infos about who performed it

NOTE: always pull before committing



Push

Step 3.

git push origin <branchname> #uploads the modifications to the desired branch

git push origin master #uploads the modifications to the master branch

git push --all origin #uploads all the modifications

Git push actually uploads the local changes to the remote repository.

Until you do not push, your colleagues will not be able to see your changes.



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

What you have to do for Web App Dev

In general, the steps that you have to perform when developing your web application are:

1. clone
2. pull (and solve conflicts, if present)
3. add (if you created or modified files)
4. commit (pull just before commit to avoid conflicts)
5. push

`git clone`

`git pull`

`git add .`

`git commit -m "message"`

`git push origin <branch>`

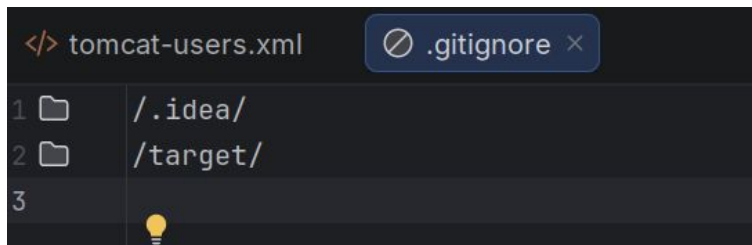


UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

.gitignore



```
</> tomcat-users.xml  .gitignore x
1  /.idea/
2  /target/
3  
```

A gitignore file specifies intentionally untracked files that Git should ignore. Files already tracked by Git are not affected.

Each line in a gitignore file specifies a pattern.
For example, you want to avoid tracking files in preliminary examples.



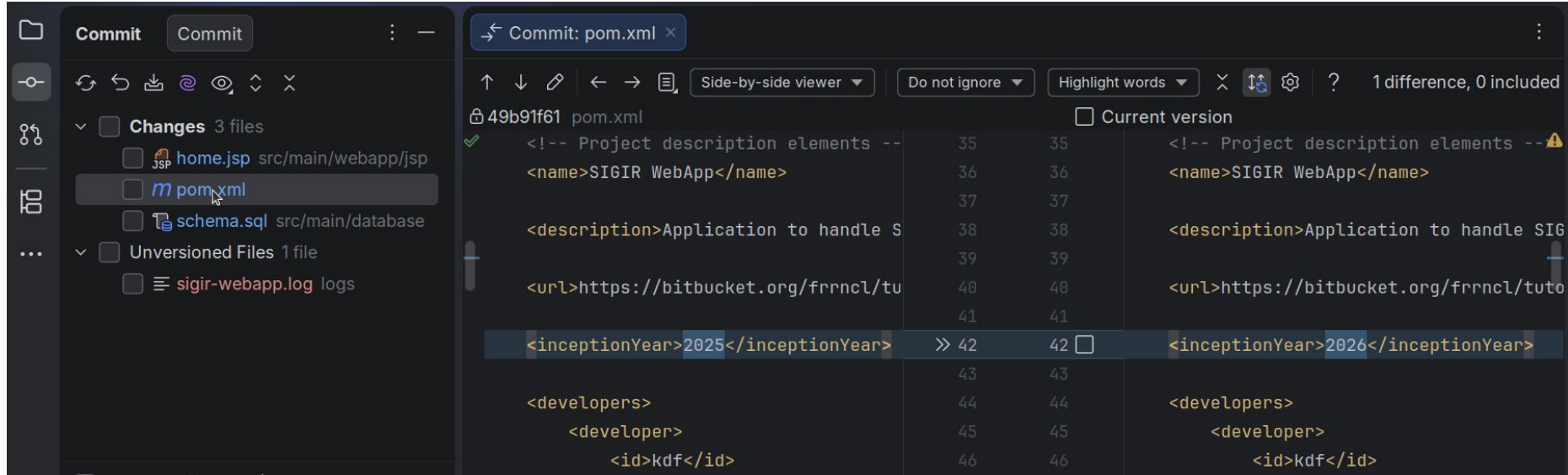
Tips

If you use separate branches (e.g. one for each group member) **do not merge them only at the end**, otherwise many conflicts may arise. **Try to merge them each time that you implement something significant.**

- Pull each time when you start working **AND** before performing a commit. This will reduce the number of conflicts and will make it easier to solve them.
- When you clone, if the repository is private you will be required to provide your credentials. We suggest to use App Passwords:
<https://support.atlassian.com/bitbucket-cloud/docs/app-passwords/>



Git on IDE (IntelliJ)



Maven



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Maven

Apache Maven is a **build automation and project management tool** used primarily for Java projects. It provides a standard way to build and manage Java-based projects and their dependencies.

Maven uses a **Project Object Model (POM)** file to describe the project and its **dependencies**, as well as the build process and any plugins that are needed. It manages dependencies and builds the project, including compiling source code, packaging artifacts, running tests, and generating documentation.

Maven also provides a set of standard lifecycle phases, such as **compile**, **test**, **package**, and **install**, which define a common build process for all Maven-based projects. This helps ensure consistency across projects and makes it easier for developers to work with different ones.



Lifecycle

- **clean:** Cleans the project by removing any previously generated build artifacts, such as compiled classes and packaged files.
- **validate:** Validates the project configuration, ensuring that it is correct and all necessary information is available.
- **compile:** Compiles the project's source code into bytecode, typically stored in the target/classes directory.
- **test:** Runs the project's tests using a suitable testing framework.



Lifecycle

- **package:** Packages the compiled code and resources into a distributable format, such as a JAR, WAR, or ZIP file.
- **verify:** Performs checks on the packaged artifact to ensure its integrity and quality.
- **install:** Installs the packaged artifact into the local Maven repository, making it available for use by other projects on the same machine.
- **deploy:** Copies the packaged artifact to a remote repository.



Dependencies

Where to find the dependencies to add to your pom.xml?

Maven Repository: <https://central.sonatype.com/>



API Doc Help  Browse Sign In

Find OSS Components

As stewards of Central for nearly 20 years and Inventors of both software supply chain management and Nexus Repository, Sonatype knows that the Integrity of your build is critical.

 Search

[Advanced Options](#)



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Before starting

Remember that you can change the folder in which maven stores the dependencies indicated in the pom.xml

To do so you just need to change the file **settings.xml** in the **.m2** folder (in your home)

If you do not have the **settings.xml** file in your **.m2** folder you can copy, paste and modify the following:

<https://bitbucket.org/frrncl/webapp-unipd/src/master/maven-setup/>



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

pom.xml

The **pom** file is an **xml** file that contains configuration information about the project and its dependencies.

- **Project information:** such as groupid, artifactid, project and version.
- **Dependencies:** external libraries or modules that the project relies on. Maven uses this information to download the necessary libraries from remote repositories and include them in the project's build process.
- **Build configuration:** how the project should be built. It includes settings for compiling source code, running tests, packaging the project into distributable formats.
- **Plugins:** Maven plugins are extensions that provide additional functionality to the build process.



Example

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unipd.dei.webapp</groupId>

  <artifactId>sigir-webapp</artifactId>

  <version>1.00</version>

  <packaging>war</packaging>

  <!-- Project description elements -->
  <name>SIGIR WebApp</name>

  <description>Application to handle SIGIR's papers and presentations</description>

  <url>https://bitbucket.org/frnnc1/tutor-wa-2025/</url>

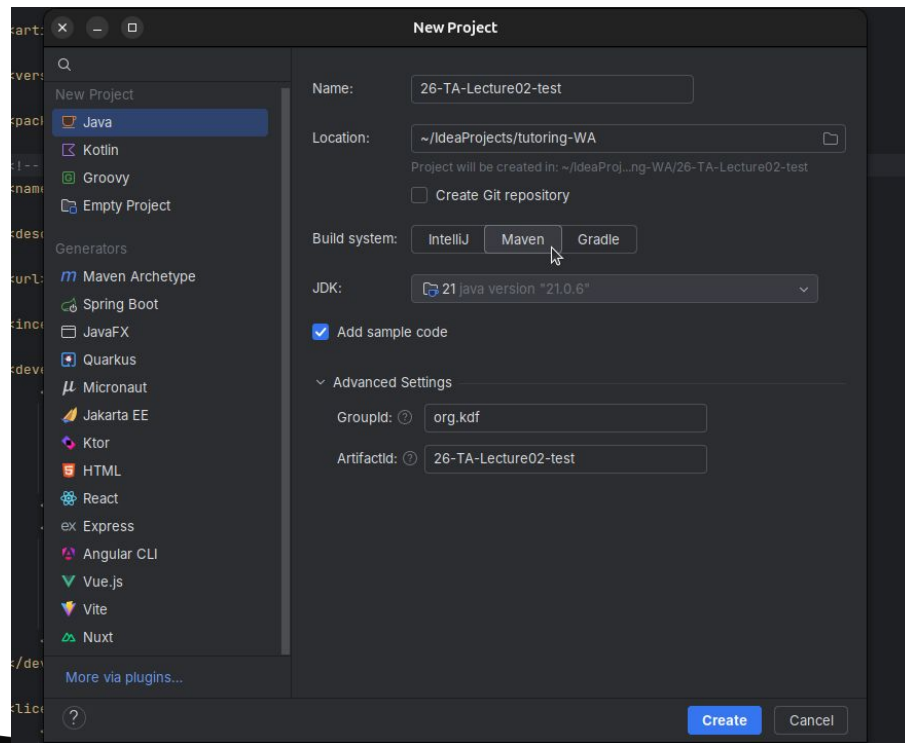
  <inceptionYear>2026</inceptionYear>

  <developers>
    <developer>
      <id>kdf</id>
      <name>Francesco Luigi De Faveri</name>
      <email>francescoLuigi.defaveri@phd.unipd.it</email>
      <url>http://www.dei.unipd.it/~defaverifr/</url>
    </developer>
```



Setting Up Project in IntelliJ

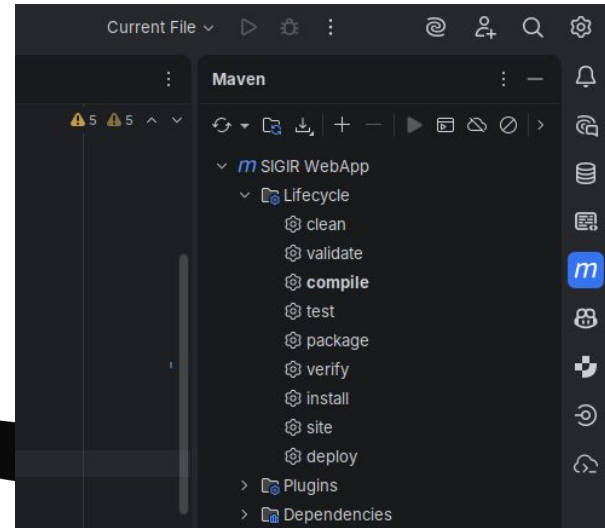
NOTE: if you start from an existing project (e.g., **professor's examples**) when you open it the first time a message will appear asking you if IntelliJ needs to open the project as a maven project.



Usage

Once installed maven, you can use it:

- From the command line: place yourself in the project folder and run `mvn clean package javadoc:javadoc`
- From **IntelliJ**, open the right Maven panel and select the phase you want to run.



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Tomcat



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Setup

Tomcat is a **web server** you use to deploy the war file. Apache Tomcat is an open-source web server and servlet container for running Java applications.

Download:

<https://tomcat.apache.org/download-11.cgi>

Suggestion: Follow the procedure for Linux/MacOS also for Windows

11.0.18

Please see the [README](#) file for packaging information. It explains what every distribution contains.

Binary Distributions

- Core:
 - [zip \(pgp, sha512\)](#)
 - [tar.gz \(pgp, sha512\)](#)
 - [Windows.zip \(pgp, sha512\)](#)
 - [Windows Service Installer \(pgp, sha512\)](#)
- Full documentation:
 - [tar.gz \(pgp, sha512\)](#)
- Deployer:
 - [zip \(pgp, sha512\)](#)
 - [tar.gz \(pgp, sha512\)](#)
- Embedded:
 - [tar.gz \(pgp, sha512\)](#)
 - [zip \(pgp, sha512\)](#)

Source Code Distributions

- [tar.gz \(pgp, sha512\)](#)
- [zip \(pgp, sha512\)](#)



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Setup

Once downloaded:

1. Unzip the downloaded folder
2. Go to %TOMCAT_HOME%/conf and open the “tomcat-users.xml” file in my case
%TOMCAT_HOME% = kdf/Download/apache-tomcat-11.0.18



Setup

3. “Create” **your** account.

```
<user username="kdf" password="kdf" roles="admin-gui,manager-gui"/>
</tomcat-users>
```

Roles:

manager-gui: allows access to the HTML GUI and the status pages

manager-script: allows access to the text interface and the status pages

manager-jmx: allows access to the JMX proxy and the status pages

manager-status: allows access to the status pages only

Suggestion: create your account with the role “manager-gui”



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Setup

4. Open a terminal in the folder %TOMCAT_HOME%/bin and run

chmod +x *.sh

5. To start the server: Open a terminal in the folder %TOMCAT_HOME%/bin and run

./startup.sh

6. To stop the server: Open a terminal in the folder %TOMCAT_HOME%/bin and run

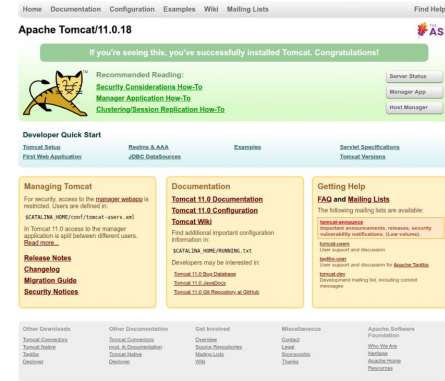
./shutdown.sh



If everything went ok

To start using tomcat:

- Open your browser and go to <http://localhost:8080/>
- Click on manager App
- Login with the credentials you set in the tomcat-users.xml file



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

After Logging...



Tomcat Web Application Manager

Message: OK

Manager

List Applications [HTML Manager Help](#) [Manager Help](#) [Server Status](#)

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

Context Path:

Version (for parallel deployment):

XML Configuration file path:

WAR or Directory path:

[Deploy](#)

WAR file to deploy

Select WAR file to upload [Choose File](#) No file chosen

[Deploy](#)



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Docker



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



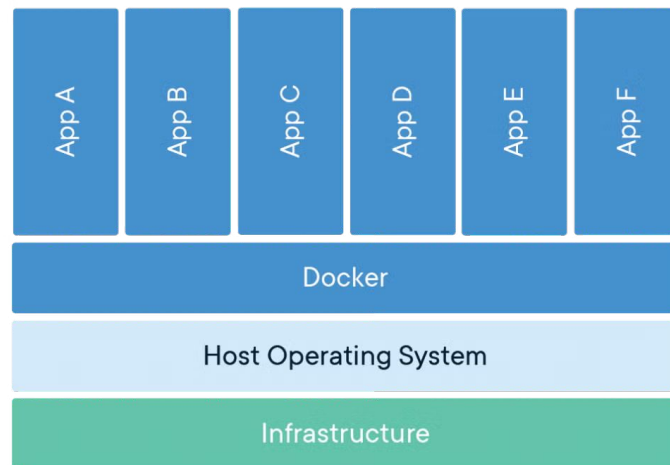
DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Containerization

Containerizing means creating a software container, which is an isolated and self-sufficient environment where an application can run consistently, regardless of the operating system or environment in which it is executed.

Advantages:

- Portability
- Efficiency
- Scalability
- Isolation



Dockerfiles & Images

The dockerfiles are the recipes to create the docker images.

Remember that docker images are immutable.

```
FROM python:3.12

WORKDIR /usr/local/app

# Install the application dependencies

RUN pip install pandas

RUN pip install scikit-learn
```

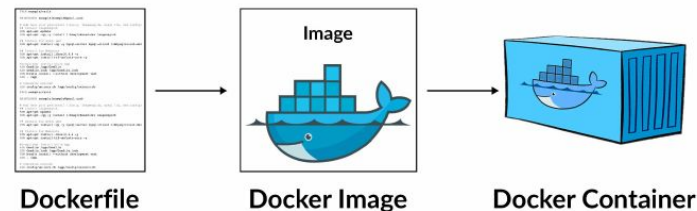


Docker Containers

Docker containers can be seen as running instances of the docker images.

A docker container may rely on multiple images.

The same image can be shared by multiple containers within the same host.



Docker Volumes

A **Docker volume** is a **mechanism for persistently storing data** generated by and accessed by the docker containers. A volume provides a way to share and manage data between the host machine and the containers.

Docker volumes are stored **outside** the container can be used to store databases, configuration files, logs, and any other data that **needs to persist beyond the lifecycle of a container**.



Download

Download links & installation guidelines:

Docker: <https://docs.docker.com/get-docker/>

Docker compose: <https://docs.docker.com/compose/>



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Docker - In Our Case

Each service runs one image

```
1 services:
2
3   web:
4     image: tomcat:latest
5     ports:
6       - "8081:8080"
7     depends_on:
8       db:
9         condition: service_healthy
10    volumes:
11      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
12      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
13      - ./logs:/usr/local/tomcat/webapps/my-logs
14      - ./tmp/context.xml:/usr/local/tomcat/webapps/dist/manager/META-INF/context.xml
15      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
16
17   db:
18     image: postgres
19     ports:
20       - "5433:5432"
21     environment:
22       - POSTGRES_PASSWORD=postgres
23       - POSTGRES_USER=postgres
24     volumes:
25       - ./data/db:/var/lib/postgresql/data/
26       - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-init.sql
27       - ./src/main/database/sigirwebapp-insert.sql:/docker-entrypoint-initdb.d/02-data.sql
```

This is a **docker-compose.yml** file



Docker - In Our Case

Each service runs one image

Two services
web & db

```
1  services:
2
3  web:
4    image: tomcat:latest
5    ports:
6      - "8081:8080"
7    depends_on:
8      db:
9        condition: service_healthy
10   volumes:
11     - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
12     - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
13     - ./logs:/usr/local/tomcat/webapps/my-logs
14     - ./tmp/context.xml:/usr/local/tomcat/webapps/dist/manager/META-INF/context.xml
15     - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
16
17 db:
18   image: postgres
19   ports:
20     - "5433:5432"
21   environment:
22     - POSTGRES_PASSWORD=postgres
23     - POSTGRES_USER=postgres
24   volumes:
25     - ./data/db:/var/lib/postgresql/data/
26     - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-init.sql
27     - ./src/main/database/sigirwebapp-insert.sql:/docker-entrypoint-initdb.d/02-data.sql
28   healthcheck:
29     test: [ "CMD-SHELL", "pg_isready -U postgres" ]
30     interval: 5s
```



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Docker - In Our Case

```
services:
  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps/dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml

  db:
    image: postgres
    ports:
      - "5433:5432"
    environment:
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_USER=postgres
    volumes:
      - ./data/db:/var/lib/postgresql/data/
      - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-init.sql
      - ./src/main/database/sigirwebapp-insert.sql:/docker-entrypoint-initdb.d/02-data.sql
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U postgres" ]
      interval: 5s
```

Tomcat will host our web application

Postgres will hold the data of our web application



Docker - In Our Case

The image is tomcat:latest

- Tomcat will host our web application
- We map port 8080 within the container to our 8081 local port

```
services:
  web:
    image: tomcat:latest
    ports:
      - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/aj-log
      - ./tmp/context.xml:/usr/local/tomcat/webapps/dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
```

We set a volume, in this case, a file.

Pattern: **local_path:container_path**

The file will be copied from the local directory to the container internal directory structure.

```
volumes:
  - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
```



Docker - In Our Case

The image is **postgres**

We map the port 5432 within the container to our 5433 local port

Setting environment variables. In this case, for postgres authentication

We set a volume, in this case, a **directory**. Files will be copied from the local directory to the container directory and the other way around.

```
db:
  image: postgres
  ports:
    - "5433:5432"
  environment:
    - POSTGRES_PASSWORD=postgres
    - POSTGRES_USER=postgres
  volumes:
    - ./data/db:/var/lib/postgresql/data/
    - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-init.sql
    - ./src/main/database/sigirwebapp-insert.sql:/docker-entrypoint-initdb.d/02-data.sql
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U postgres" ]
    interval: 5s
    timeout: 10s
    retries: 50
```



Docker - In Our Case

Adding dependencies between services

NOTE: the condition is needed since normally the dependencies are instantiated as soon as the services are running

In our case we want the dependency to be instantiated when the **db** is ready ("healthy")

```
services:
  web:
    - "8081:8080"
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./target/sigir-webapp-1.00.war:/usr/local/tomcat/webapps/sigir-webapp.war
      - ./tmp/server.xml:/usr/local/tomcat/conf/server.xml
      - ./logs:/usr/local/tomcat/webapps/my-logs
      - ./tmp/context.xml:/usr/local/tomcat/webapps/dist/manager/META-INF/context.xml
      - ./tmp/tomcat-users.xml:/usr/local/tomcat/conf/tomcat-users.xml
  db:
    image: postgres
    ports:
      - "5433:5432"
    environment:
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_USER=postgres
    volumes:
      - ./data/db:/var/lib/postgresql/data/
      - ./src/main/database/schema.sql:/docker-entrypoint-initdb.d/01-init.sql
      - ./src/main/database/sigirwebapp-insert.sql:/docker-entrypoint-initdb.d/02-data.sql
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U postgres" ]
      interval: 5s
      timeout: 10s
      retries: 50
```

Health check to understand when the db is up and running



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Docker - In Our Case

What if we also want to initialize our database starting from a “.sql” file that contains our database schema?

We need to add a new volume to the db service:

- `./data/schema.sql:/docker-entrypoint-initdb.d/init.sql`

Important notes:

- **Pay attention to the paths**, they are relative to the folder which contain your docker-compose.yml file
- In the shown example the we mapped the **container “standard” ports to different local ports**. This is needed because the “standard” ports might be already used (e.g., if you already installed postgres locally).



Docker Commands

Execute these command in the same folder containing the yml file

- *creating and starting the container*
docker compose up
- *stopping the container*
docker compose down
- *list the containers*
docker ps
- *Connect to DB in a container, if the container name is <my_container>*
Docker exec -it my_container psql -U postgres



Final Remark on HW1



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

README in the Group Repository

Your bitbucket repository should contain a README.md file that describes the main structure and contents.

These information include:

- **Installation and setup procedures**, main features of the software, contributors (I suggest you to also report here the contributions of the report, for the **features** of the web app try to **summarise** in **few lines**, and **be specific on the installation and setup**)

To write the README you should use the markdown language which is quite simple and intuitive.

You can find the syntax here: <https://www.markdownguide.org/basic-syntax/>



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE